GOVERNMENT ENGINEERING COLLEGE • TECHNICAL ARCHITECTURE

Campus Voice — Comprehensive Technical Guide

Full-Stack Architecture, Technology Rationale, and Viva/Interview Reference

Target System: Institutional Announcement & Peer Q&A Portal

Stack: Vanilla JS, HTML5, Tailwind CSS, PDF.js, Node.js Native HTTP

Port: 3000

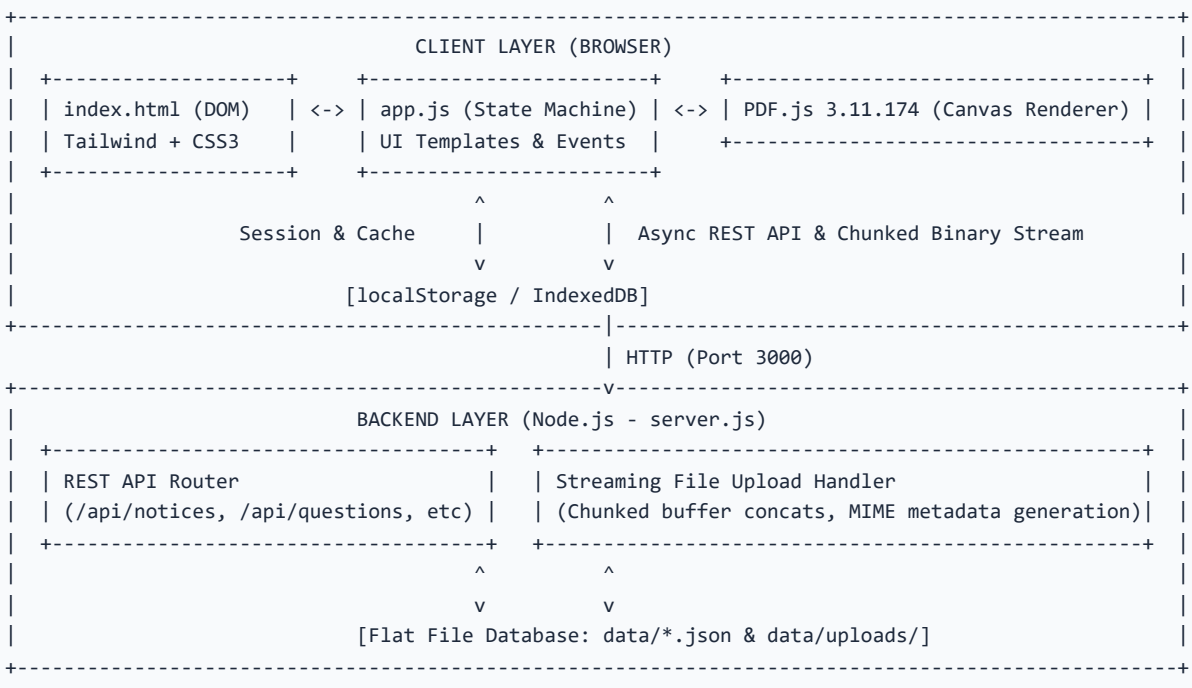
1. Executive Summary & Problem Solved

In engineering colleges, critical communications (exam circulars, lab timetables, on-campus drive schedules, hackathon calls) are typically distributed through unorganized WhatsApp groups, unofficial social channels, or physical bulletin boards. This causes circular loss, unverified rumors, and communication friction.

Campus Voice solves this with a unified, dual-module institutional web portal:

- **Official Notice Board:** Faculty-published circulars with branch targeting (CSE, AIML, ECE, ME, CV), category filtering, upvoting, bookmarking, and high-fidelity PDF/image attachment previewing.
- **Student Community Q&A:** A moderated peer-to-peer discussion forum (StackOverflow/Reddit style) categorized into topics (Placements, Exams, Projects, Academics, Hostel), featuring verified faculty responses, accepted answer resolution, anonymous posting, and community reporting.

2. High-Level Architectural Flow



3. Full Technology Breakdown: What, Why, and How

Technology	Why We Chose It (Rationale)	How It Is Implemented (Code Details)
Vanilla JavaScript ES6+ (Client)	**Zero build step and maximum performance.** No framework overhead (React/Vue/Angular), instant page boot, minimal memory consumption, and straightforward debugging directly in browser DevTools.	`app.js` manages reactive client state machines (`currentRole` , `activeTab` , `currentFilter`), builds dynamic UI nodes via template literals, enforces XSS sanitization, and handles asynchronous API calls.
HTML5 & Semantic DOM	Provides accessible landmarks (`<header>` , `<nav>` , `<main>` , `<dialog>`), fast parsing, clear hierarchy, and natural keyboard navigability.	`index.html` defines structured sections (`#noticesSection` , `#communitySection`), accessible skip links, aria attributes, and modal backdrops.
Tailwind CSS (CDN Version)	Offers utility-first styling with consistent spacing rhythms (4px/8px), fluid typography, and built-in responsive breakpoints (`sm:` , `md:` , `lg:`) without authoring hundreds of media queries.	Configured inline with customized color tokens: `brand` (50–950 indigo spectrum), `accent` (amber spectrum), `success` , `warning` , `danger` , and custom elevation shadows.
Vanilla CSS3 (style.css)	CDN Tailwind cannot dynamically target template-string-generated HTML or enforce strict CSS inheritance rules. Custom CSS gives granular, low-level styling control.	Defines `:root` and `.dark` CSS variables, strict Role-Based Access Control rules, modal entrance keyframes (`modalPanelIn`), micro-interactions (`.btn-press` , `.upvote-pop`), and custom scrollbars.
PDF.js 3.11.174 (Mozilla)	Standard browsers force PDF downloads or launch external readers. PDF.js allows direct, in-app vector canvas rendering without third-party plugins.	Loads raw binary array buffers, executes viewport calculations with device-pixel-ratio scaling for retina screens, and draws pages onto `<canvas>` elements inside `#fileViewerModal` .
Node.js Native HTTP (server.js)	**Zero external npm dependencies.** Eliminates Express/Fastify bloat, runs on only ~20 MB RAM, and is immune to third-party supply chain vulnerabilities.	Uses Node's built-in `http` , `fs` , and `path` modules. Implements a custom REST router, CORS handler, static asset server, and chunked streaming binary uploader.
Flat-File JSON DB (data/*.json)	Simplicity, portability, and zero configuration. Eliminates the requirement to set up, authenticate, and manage external database servers (Postgres/Mongo) for local deployment.	Stores normalized JSON files: `notices.json` , `questions.json` , `answers.json` , `users.json` , `categories.json` . The server executes atomic reads/writes on every mutation.
Browser Web Storage	Maintains persistent client sessions, instantaneous offline hydration, and user preferences across tabs.	`localStorage` stores student session credentials (`campusSession`), theme choice (`campusTheme`), bookmarks, and upvote maps. `IndexedDB` caches binary attachment files for offline viewing.

4. Core Architectural Systems: Deep Dives

A. Role-Based Access Control (RBAC) via CSS Hierarchy

Rather than executing heavy DOM manipulation loops to query and hide role-specific buttons on every render, role visibility is handled directly by the browser's CSS rendering engine:

> **The Strict Role Display Architecture:**
>
> ```
> .faculty-only { display: none !important; }
> body.role-faculty .faculty-only { display: block !important; }
> .student-only { display: none !important; }
> body.role-student .student-only { display: block !important; }
> ```

When a user logs in, JavaScript simply executes `document.body.classList.add("role-faculty")`. The browser instantly reflows and displays faculty controls (e.g., "Post Notice", "Delete", "Add Category") while keeping students restricted. This architecture makes role transitions instantaneous and impossible to break via accidental inline style changes.

B. XSS Defense & Input Sanitization

Because notices and Q&A threads are injected dynamically into the DOM using template literals, the application incorporates a rigorous Cross-Site Scripting (XSS) defense layer:

- `escapeHtml(str)`: Replaces `&`, `<`, `>`, and `"` with HTML entities to neutralize script injection.
- `escapeJs(str)`: Neutralizes quotes and special characters within inline event handler arguments (e.g., `onclick="openFileViewer('${escapeJs(fileId)}')"`).

C. Flash-of-Unstyled-Content (FOUC) Dark Mode Architecture

To prevent the jarring "white flash" that occurs when dark mode is initialized after CSS loads, a synchronous, blocking script is placed directly in the `<head>` of `index.html`:

> ```
> const saved = localStorage.getItem("campusTheme");
> const prefersDark = window.matchMedia("(prefers-color-scheme: dark)").matches;
> if (saved === "dark" || (!saved && prefersDark)) document.documentElement.classList.add("dark");
> ```

This executes before the browser paints its initial frame, ensuring seamless dark mode rendering from millisecond zero.

D. Chunked Streaming Binary File Uploads

Unlike frameworks requiring third-party libraries like Multer, `server.js` natively consumes binary streams:

1. Incoming `POST /api/files/:id` requests stream raw data chunks into memory.
2. `Buffer.concat(chunks)` stitches the payload together into a single atomic buffer.
3. The buffer is written directly to `data/uploads/{fileId}` while headers (`X-File-Name`, `X-File-Type`) are persisted to a companion JSON metadata file (`{fileId}.meta.json`).
4. When requested via `GET /api/files/:id`, the server sets correct `Content-Type` and `Content-Disposition: inline` headers, allowing PDF.js or browser image decoders to display the file natively.

5. Viva / Technical Interview Q&A Cheat Sheet

Q1: Why didn't you use React, Next.js, or Vue for this project?

Answer: "For an institutional campus intranet, Vanilla JavaScript provides zero build dependencies, instant startup, sub-50ms load times, and a near-zero memory footprint. It avoids build-tool vulnerabilities, npm supply-chain bloat, and framework deprecation cycles while giving complete mastery over the DOM."

Q2: Why did you build the server with Node's native HTTP module instead of Express?

Answer: "By avoiding Express, our backend has zero third-party dependencies and runs entirely on Node.js core. This guarantees negligible memory usage (~20 MB), lightning-fast execution, and complete elimination of npm package vulnerabilities. It also showcases fundamental knowledge of HTTP protocols, streams, and file I/O."

Q3: How does the in-app PDF preview work without forcing a file download?

Answer: "We integrated Mozilla's PDF.js library. When a user clicks 'View Attachment', the client fetches the binary PDF as an ArrayBuffer. PDF.js decodes the PDF's vector stream and renders each page directly onto an HTML5 `<canvas>` element, calculating device-pixel-ratio scaling for sharp rendering on retina screens."

Q4: How does the system handle multi-browser synchronization?

Answer: "The server enforces CORS headers (`Access-Control-Allow-Origin: *`). If a faculty member on Chrome posts a circular or accepts a student's answer, it is atomically committed to the backend JSON store. When a student on Edge or a mobile browser loads or refreshes their feed, the updated state is immediately fetched and reflected."

Q5: How would you scale this architecture to support 10,000+ simultaneous students?

Answer: "1. Replace flat JSON storage with an indexed database like PostgreSQL (with connection pooling) or MongoDB.
2. Offload uploaded file storage from local disk to AWS S3 or Cloudflare R2 behind a CloudFront CDN.
3. Implement JWT-based authentication with bcrypt-hashed passwords.
4. Integrate WebSockets (or Server-Sent Events) to push real-time circular notifications without manual refreshes."